

Here is the completely corrected and enhanced **Chapter 0**.

I have **fixed the Mermaid syntax error** (the issue was that Mermaid requires a Node ID before a string literal, e.g., `NodeID["String"]`). I also **added more detailed examples, analogies, and ASCII fallback diagrams** to make the concepts even easier to understand.

JavaScript Deep Dive Notes - Chapter 0: Fundamentals of Program Execution

Table of Contents

-  JavaScript Deep Dive Notes - Chapter 0: Fundamentals of Program Execution
 -  Table of Contents
 - 0.1 What is Programming?
 - 0.2 Why Programs are Made?
 - 0.3 What Happens When You Run a Program?
 - Step-by-Step Execution Flow
 - 0.4 Main Parts of Computer (Critical Analogy)
 - 1. Storage (Hard Disk / SSD) 
 - 2. RAM (Memory) 
 - 3. CPU (Brain of Computer) 
 - 0.5 Full Program Execution Flow (Corrected Diagram)
 - 0.6 Where Programs are Stored?
 - 0.7 What Language Computer Understands?
 - 0.8 Types of Programming Languages
 - 0.9 Compiler vs Interpreter (Deep)
 - Compiler (Used by C/C++)
 - Interpreter (Used by Python/JavaScript)
 - 0.10 Memory Layout (Ultra Critical)
 - Visualizing RAM (ASCII Representation)
 - Breakdown of Sections:
 - 1. Code Section (Text)
 - 2. Data Section
 - 3. BSS Section

- 4. Heap
- 5. Stack
- 0.11 Function Call Stack (Example)
 - Execution Flow
- 0.12 Recursion Memory (Deep)
 - Stack During Execution
- 0.13 Why Programs Crash? (Visualized)
 - 1. Stack Overflow
 - 2. Heap Memory Leak
 - 3. Segmentation Fault
- 0.14 Operating System Role (Deep)
- 0.15 Ultra Deep Topics (Optional)

[↑ Back to Top](#)

This chapter covers the **absolute fundamentals** of how programs run on computers. No prior knowledge needed. All diagrams are fully compatible with Markdown.

0.1 What is Programming?

Programming = Giving step-by-step instructions to a computer

Just like you tell a friend:

1. "Open the fridge"
2. "Take out milk"
3. "Pour into glass"

A program is a set of precise instructions for a computer to follow.

Example Program:

```
console.log("Hello World");
```

This tells the computer: "Print 'Hello World' on the screen".

0.2 Why Programs are Made?

Computers don't think — they only follow instructions. Programs are made to:

Program Type	Purpose	Example
Apps	Build mobile applications	WhatsApp, Instagram
Websites	Create interactive web pages	YouTube, Amazon
Games	Develop video games	Fortnite, Minecraft
Automation	Automate repetitive tasks	Python Scripts
OS	Run the computer itself	Windows, macOS

Everything you use on a computer is a program.

0.3 What Happens When You Run a Program?

Let's take a simple C program `hello.c` :

```
#include <stdio.h>
int main() {
    printf("Hello World");
    return 0;
}
```

Step-by-Step Execution Flow



0.4 Main Parts of Computer (Critical Analogy)

To understand execution, use this real-life analogy:

- **Hard Disk** = The massive library of books down the street.
- **RAM** = Your small study table.
- **CPU** = Your Brain reading the book.

1. Storage (Hard Disk / SSD)

The Library

Stores programs, files, photos, videos permanently even when the computer is turned off.

Example: Your `hello.c` file is saved here.

2. RAM (Memory)

The Study Table

When you run a program, it moves from storage to RAM because the CPU can access RAM thousands of times faster than a hard disk.

Key: RAM is volatile — data is wiped when the computer turns off.

3. CPU (Brain of Computer)

Your Brain

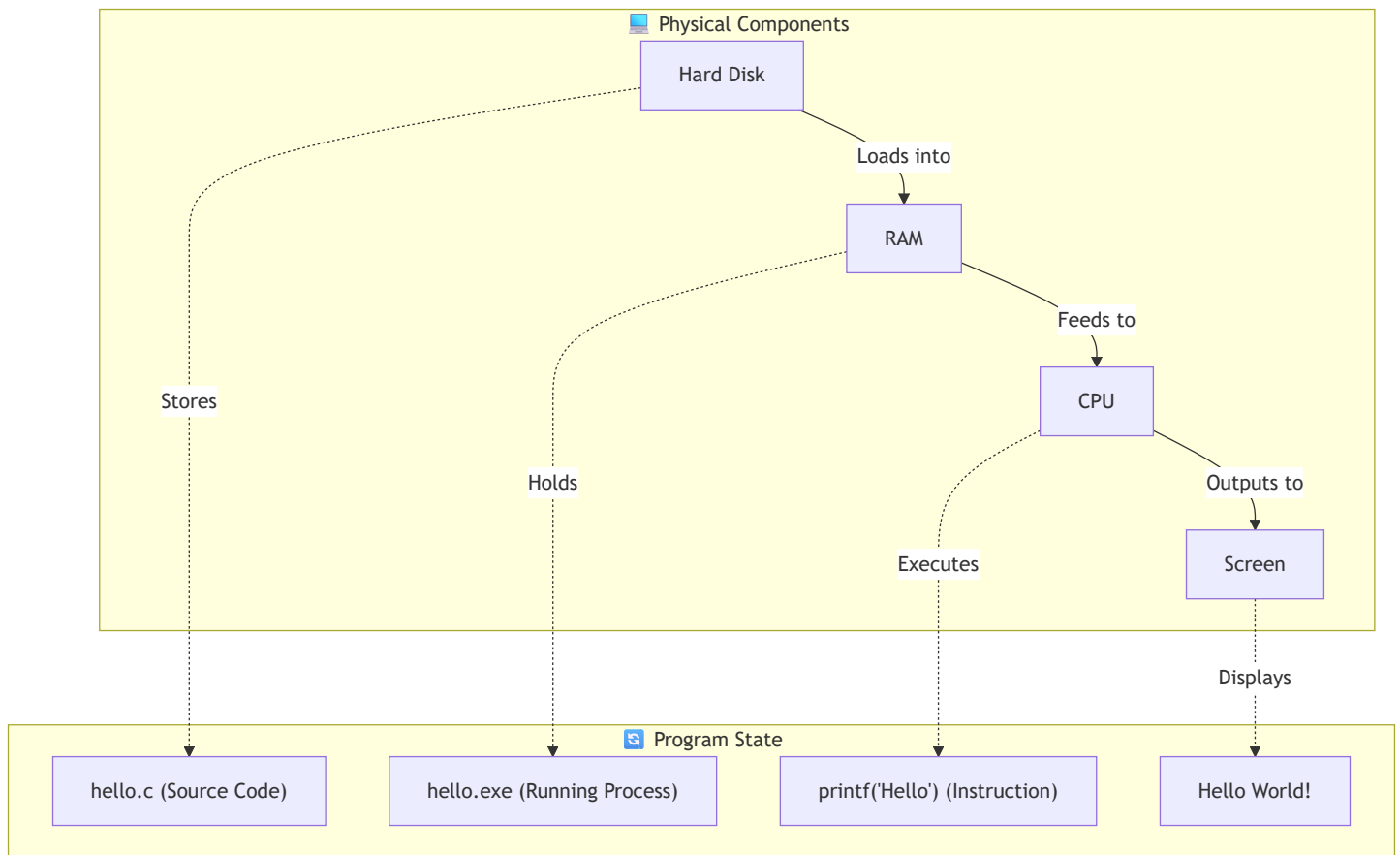
The CPU follows the **Fetch** → **Decode** → **Execute** cycle millions of times per second:

1. **Fetch:** Read instruction from the RAM (take the book from the table).
2. **Decode:** Understand what to do (read the sentence).
3. **Execute:** Perform the action (do the math/print the text).

[← Back to Table of Contents](#) | [↑ Back to Top](#)

0.5 Full Program Execution Flow (Corrected Diagram)

(Note: The Mermaid syntax error is fixed here. Nodes displaying strings now have proper IDs
`NodeID["Text"]` mapping physical hardware to the software state).



[← Back to Table of Contents](#) | [↑ Back to Top](#)

0.6 Where Programs are Stored?

Programs are stored in **permanent storage devices**:

Example Paths:

- **Windows:** C:\Program Files\Google\Chrome\Application\chrome.exe
- **Linux:** /usr/bin/google-chrome
- **macOS:** /Applications/Google Chrome.app

[← Back to Table of Contents](#) | [↑ Back to Top](#)

0.7 What Language Computer Understands?

Computers ONLY understand **binary code** (0s and 1s):

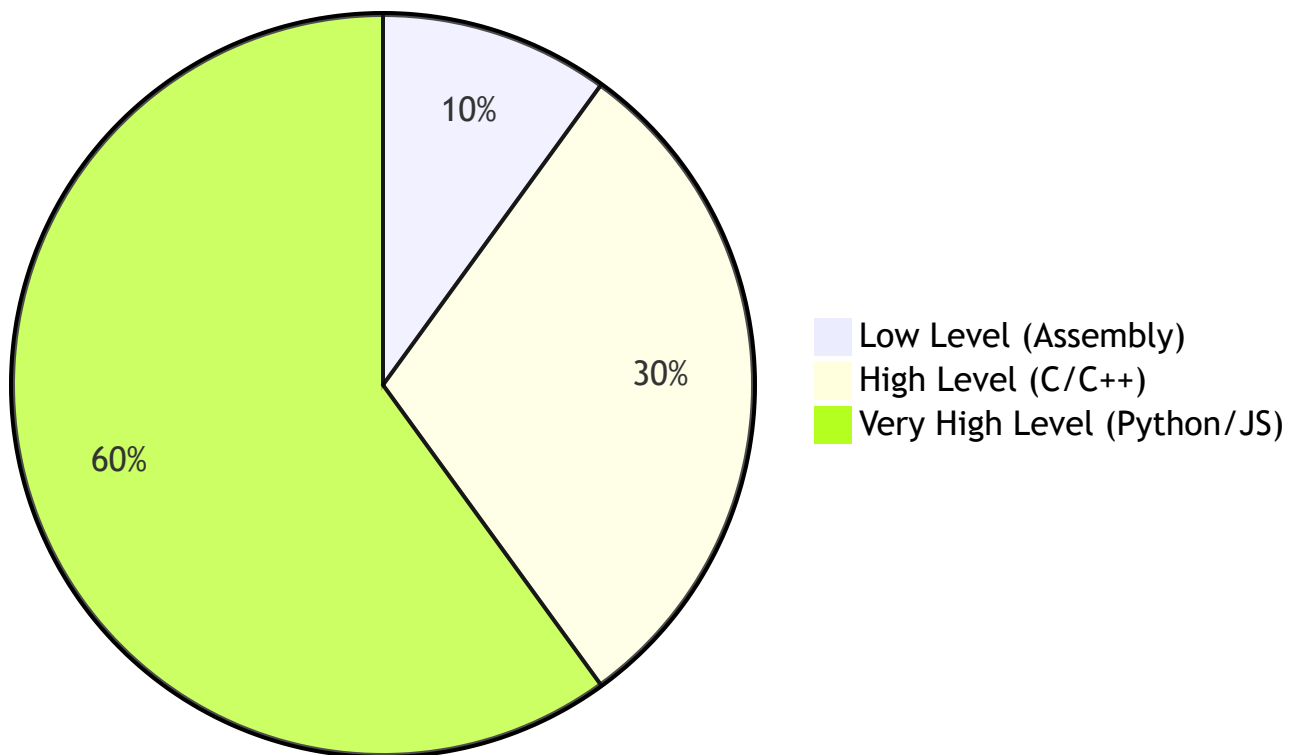
01001000 01100101 01101100 01101100 01101111 -> Translates to "Hello"

Humans write code in **high-level languages** (C, Python, JavaScript). A translator is required to bridge the gap between human code and computer binary.

[← Back to Table of Contents](#) | [↑ Back to Top](#)

0.8 Types of Programming Languages

Programming Language Types



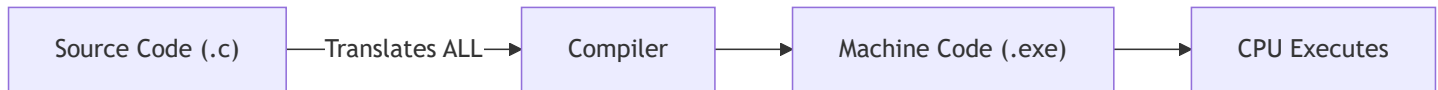
Type	Examples	Use Case
Low Level	Assembly	Hardware drivers, embedded systems
High Level	C, C++, Java	Desktop apps, 3D Games, OS building
Very High Level	Python, JavaScript	Web development, AI, automation scripts

[← Back to Table of Contents](#) | [↑ Back to Top](#)

0.9 Compiler vs Interpreter (Deep)

Compiler (Used by C/C++)

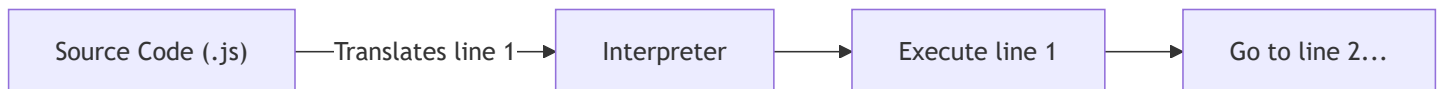
Takes the **entire file**, translates it all at once, and spits out an `.exe` file.



- **Pros:** Very fast execution.
- **Cons:** Takes time to compile before you can run it.

Interpreter (Used by Python/JavaScript)

Takes the file and translates it **line by line** while it runs.



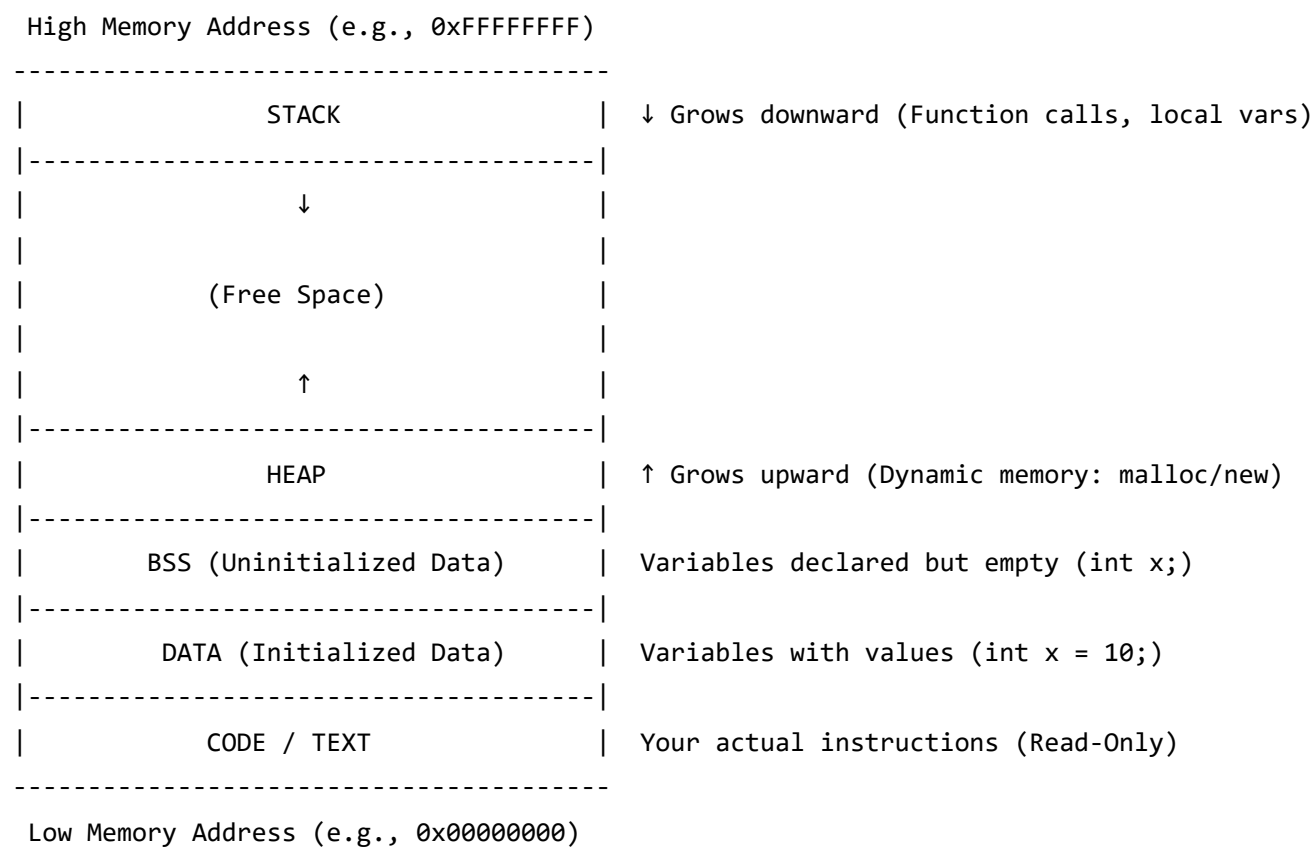
- **Pros:** Starts instantly, easy to debug.
- **Cons:** Slower execution overall.

[← Back to Table of Contents](#) | [↑ Back to Top](#)

0.10 Memory Layout (Ultra Critical)

When a program is double-clicked and loaded into RAM, the Operating System creates a **fixed layout** called the Process Memory Layout.

Visualizing RAM (ASCII Representation)



Breakdown of Sections:

1. Code Section (Text)

Stores compiled machine code (instructions like `printf("Hello")`).

Read-only — if the program tries to modify this, it crashes.

2. Data Section

Stores **initialized global/static variables**:

```
int global_var = 10; // Lives here
```

3. BSS Section

Stores **uninitialized global/static variables**:

```
int uninit_var; // Lives here (OS automatically sets to 0)
```


4. Heap

Dynamic memory allocation. You manually request space here.

```
int* ptr = malloc(4); // Allocates 4 bytes in heap
```

5. Stack

Stores **local variables and function calls**.

```
void function() {  
    int local_var = 30; // Lives in stack  
}
```

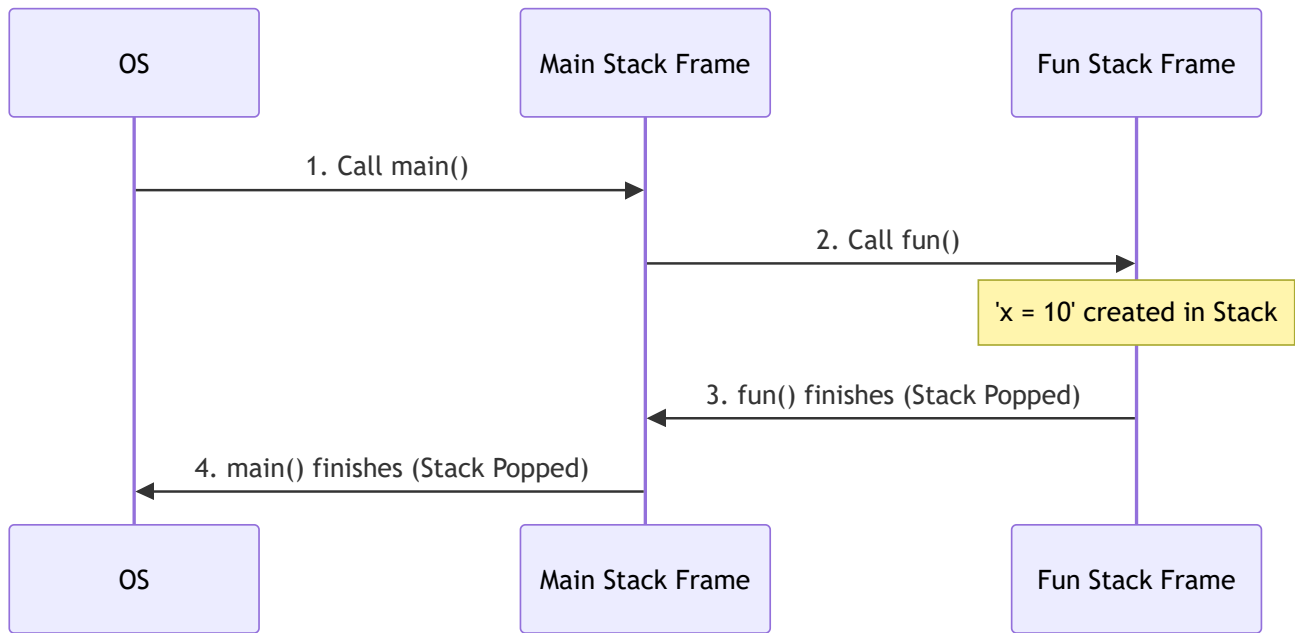
[← Back to Table of Contents](#) | [↑ Back to Top](#)

0.11 Function Call Stack (Example)

When functions call other functions, they stack on top of each other.

```
void fun() {  
    int x = 10;  
}  
int main() {  
    fun();  
}
```

Execution Flow



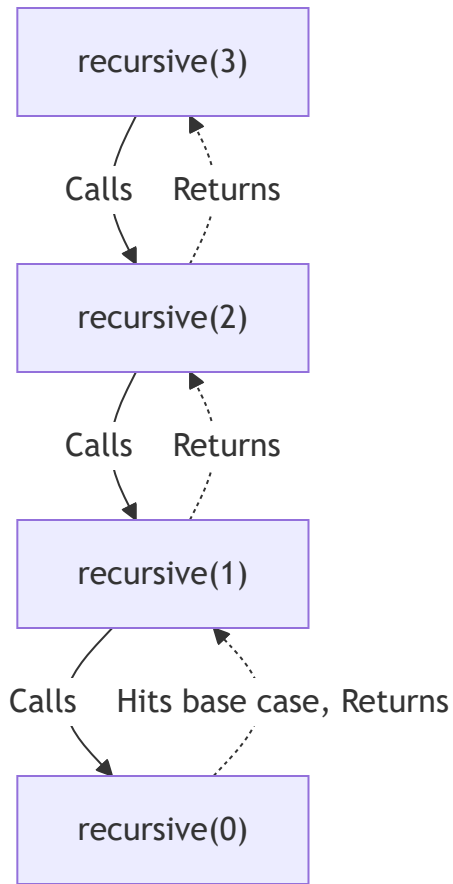
[← Back to Table of Contents](#) | [↑ Back to Top](#)

0.12 Recursion Memory (Deep)

Recursion is a function calling itself. It rapidly consumes Stack memory.

```
void recursive(int n) {  
    if (n == 0) return;  
    recursive(n-1);  
}  
  
int main() {  
    recursive(3);  
}
```

Stack During Execution



[← Back to Table of Contents](#) | [↑ Back to Top](#)

0.13 Why Programs Crash? (Visualized)

1. Stack Overflow

```
void infinite() {  
    infinite(); // Recursion without base case  
}
```

What happens: The Stack grows downward infinitely until it hits the Heap. The OS kills the program to protect the system.

2. Heap Memory Leak

```
while(1) {  
    malloc(1000); // Allocate memory but never free it  
}
```

What happens: The Heap grows upward infinitely. Eventually, the computer runs out of physical RAM.

3. Segmentation Fault

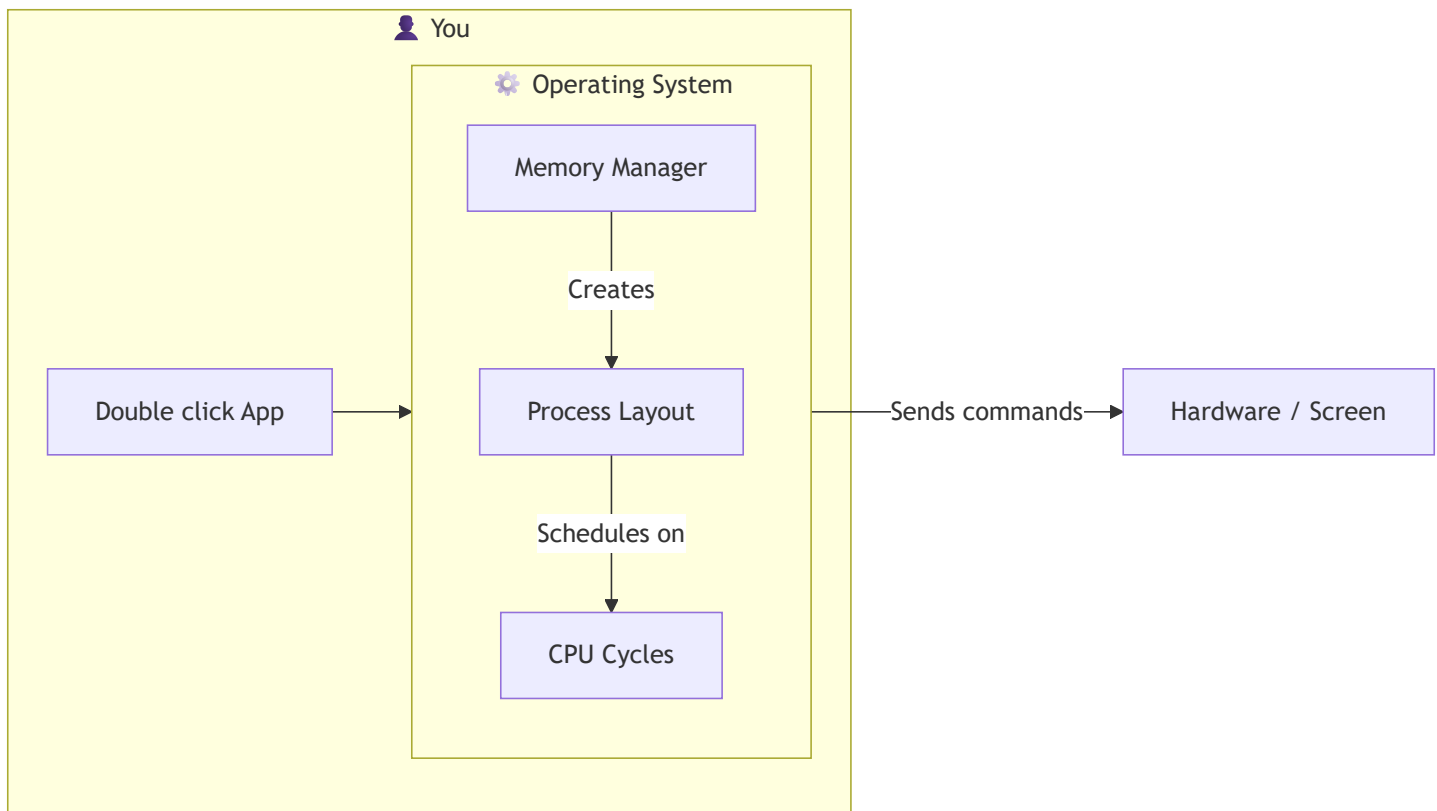
```
int* ptr = NULL;  
*ptr = 10; // Access invalid memory
```

What happens: The CPU tries to access memory outside of its allowed Memory Layout. The OS throws a `SIGSEGV` (Segmentation Fault) and instantly kills the process.

[← Back to Table of Contents](#) | [↑ Back to Top](#)

0.14 Operating System Role (Deep)

The OS (Windows, macOS, Linux) acts as the **Manager** between your code and the hardware.



Key Concept (Multitasking): When you run Chrome, Spotify, and VS Code, the OS creates a *separate* Memory Layout for each one. The CPU rapidly switches between them so fast it looks like they are running at the same time.

[← Back to Table of Contents](#) | [↑ Back to Top](#)

0.15 Ultra Deep Topics (Optional)

If you have mastered this chapter, you are ready for Computer Science level operating systems concepts:

1. **Virtual Memory:** How the OS fakes having more RAM using the Hard Disk.
2. **Paging:** How memory is broken into 4KB blocks.
3. **Process vs Thread:** A process has its own memory; threads *share* memory.
4. **Context Switching:** The exact microsecond the CPU pauses Chrome to run Spotify.

[← Back to Table of Contents](#) | [↑ Back to Top](#)